

SAP ABAP RAP – Unit 8

Exercise 20 – Validate the Flight Price

Behavior Validation using ZR_09FLIGHT / ZBP_R_09FLIGHT

Unit	Unit 8 – Describing the ABAP RESTful Application Programming Model
Exercise	20 – Validate the Flight Price
Behavior Definition	ZR_09FLIGHT
Behavior Implementation	ZBP_R_09FLIGHT / local handler LHC_ZR_09FLIGHT
Validation	validatePrice – Price must be greater than 0

1. Objective

The purpose of this exercise is to implement a RAP validation that prevents a flight from being saved when its Price is zero or negative. A valid flight price must be a positive value greater than zero.

Expected behavior

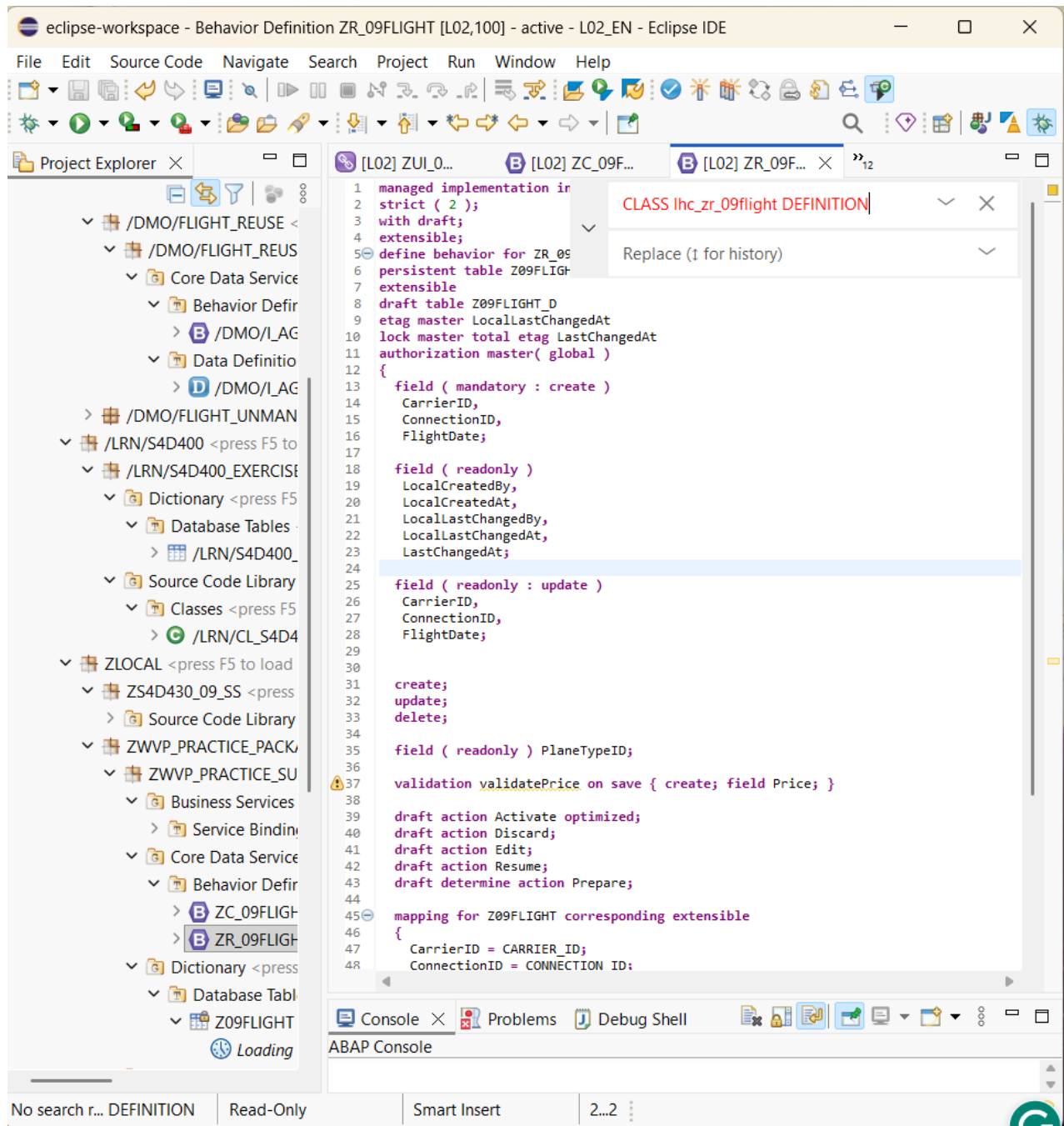
Input Price	Validation	Expected Result
5000	5000 > 0	Save succeeds
0	0 <= 0	Save rejected + error message
-100	-100 <= 0	Save rejected + error message

2. Task 1 – Define the Validation

Open the behavior definition ZR_09FLIGHT. The validation is defined with the name validatePrice and is executed during save. It runs for create unconditionally and for other operations when the Price field is changed.

```
validation validatePrice on save { create; field Price; }
```

This is the key RAP behavior-definition statement for the exercise.



Screenshot 1 – Behavior Definition ZR_09FLIGHT with validatePrice

3. Activate the Behavior Definition

Activate the behavior definition with Ctrl + F3. The validation definition must be active before the implementation can be used by the RAP runtime.

4. Create the Validation Handler Method

Place the cursor on validatePrice and use Ctrl + 1 (Quick Fix). Choose the option to add the method for the validation.

The method is created in the local handler class LHC_ZR_09FLIGHT inside the global behavior implementation class ZBP_R_09FLIGHT.

Where the method is created

```
CLASS lhc_zr_09flight DEFINITION
  INHERITING FROM cl_abap_behavior_handler.
```

The generated method has the importing parameter for the validation keys:

```
METHODS validatePrice FOR VALIDATE ON SAVE
  IMPORTING keys FOR Flight~validatePrice.
```

The actual implementation is therefore placed in METHOD validatePrice of LHC_ZR_09FLIGHT.

5. Declare the Failed and Reported Structures

At the beginning of validatePrice, create structures with the line types of failed-flight and reported-flight.

```
DATA: failed_record LIKE LINE OF failed-flight,
      reported_record LIKE LINE OF reported-flight.
```

failed_record is used to identify the instance whose validation failed. reported_record is used to return the validation message to the RAP framework.

6. Read the Transactional Buffer

Use READ ENTITIES with IN LOCAL MODE. This is important because validation works on the current transactional data, including unsaved changes in the RAP transaction.

```
READ ENTITIES OF ZR_09FLIGHT IN LOCAL MODE
  ENTITY Flight
  FIELDS ( Price )
  WITH CORRESPONDING #( keys )
  RESULT DATA(flights).
```

The key fields do not need to be listed after FIELDS because RAP automatically reads the key fields. The inline declaration creates the result table flights.

7. Loop Through the Flights

```
LOOP AT flights INTO DATA(flight).

ENDLOOP.
```

Each record is placed into the inline work area flight so its Price and key can be checked.

8. Check Whether Price Is Positive

The validation condition is Price <= 0. This catches both zero and negative values.

```
IF flight-price <= 0.

  failed_record-%tky = flight-%tky.
  APPEND failed_record TO failed-flight.

  reported_record-%tky = flight-%tky.

ENDIF.
```

For draft-enabled RAP business objects, %tky is used to carry the complete transactional key of the current instance.

9. Create the Error Message

Before appending reported_record, assign a message object using new_message. The exercise uses message class /LRN/S4D400, message number 101, with error severity.

```
reported_record-%msg = new_message(  
  id      = '/LRN/S4D400'  
  number  = '101'  
  severity = ms-error ).
```

10. Complete Validation Implementation

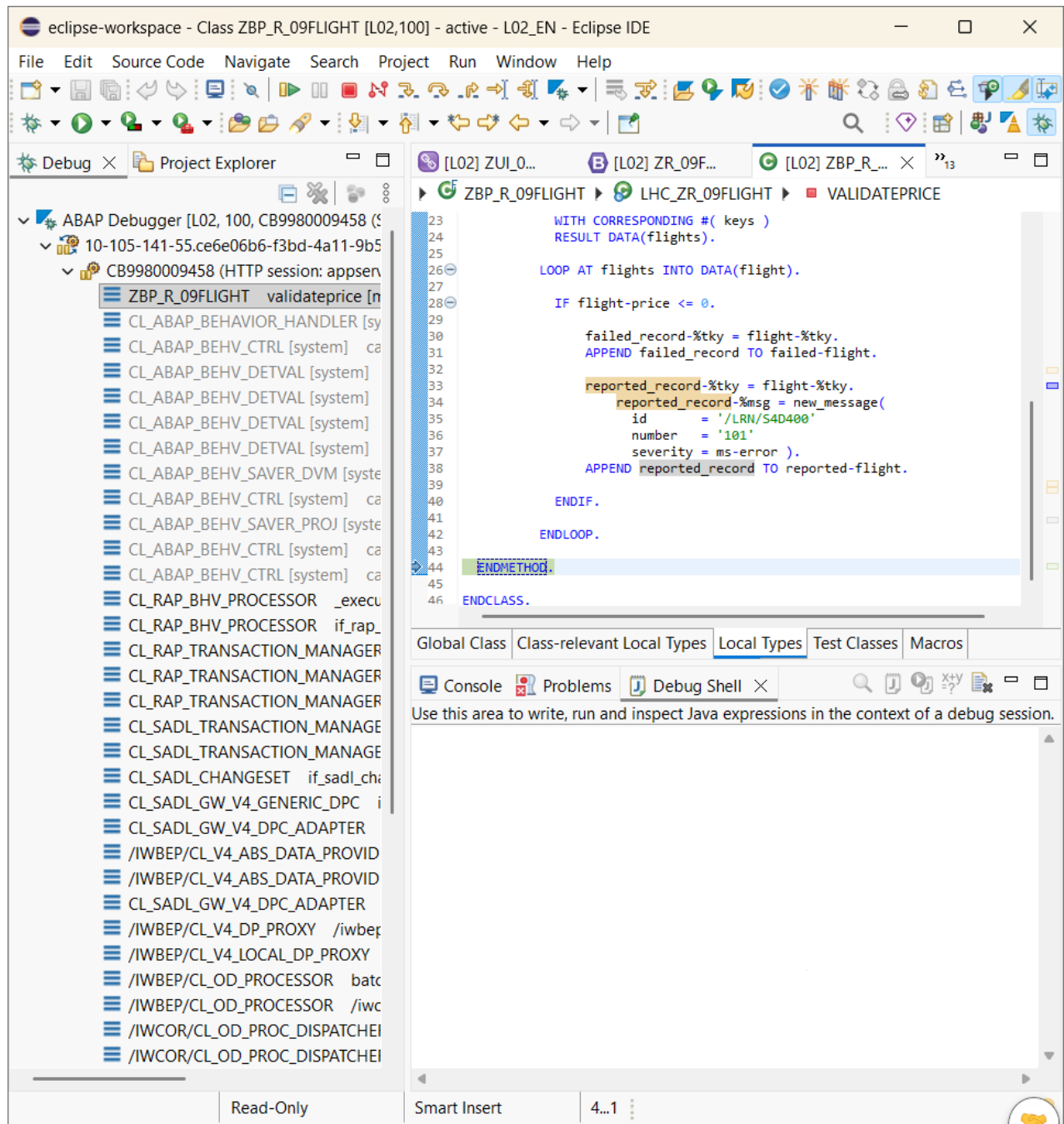
```
METHOD validatePrice.  
  
DATA: failed_record LIKE LINE OF failed-flight,  
      reported_record LIKE LINE OF reported-flight.  
  
READ ENTITIES OF ZR_09FLIGHT IN LOCAL MODE  
  ENTITY Flight  
  FIELDS ( Price )  
  WITH CORRESPONDING #( keys )  
  RESULT DATA(flights).  
  
LOOP AT flights INTO DATA(flight).  
  
  IF flight-price <= 0.  
  
    failed_record-%tky = flight-%tky.  
    APPEND failed_record TO failed-flight.  
  
    reported_record-%tky = flight-%tky.  
    reported_record-%msg = new_message(  
      id      = '/LRN/S4D400'  
      number  = '101'  
      severity = ms-error ).  
  
    APPEND reported_record TO reported-flight.  
  
  ENDIF.  
  
ENDLOOP.  
  
ENDMETHOD.
```

Activate the global class with Ctrl + F3 after completing the implementation.

11. Task 2 – Test and Debug

A breakpoint was set at the READ ENTITIES statement. The OData V4 Fiori Elements preview was restarted, an existing flight was opened, the price was changed, and Save was pressed.

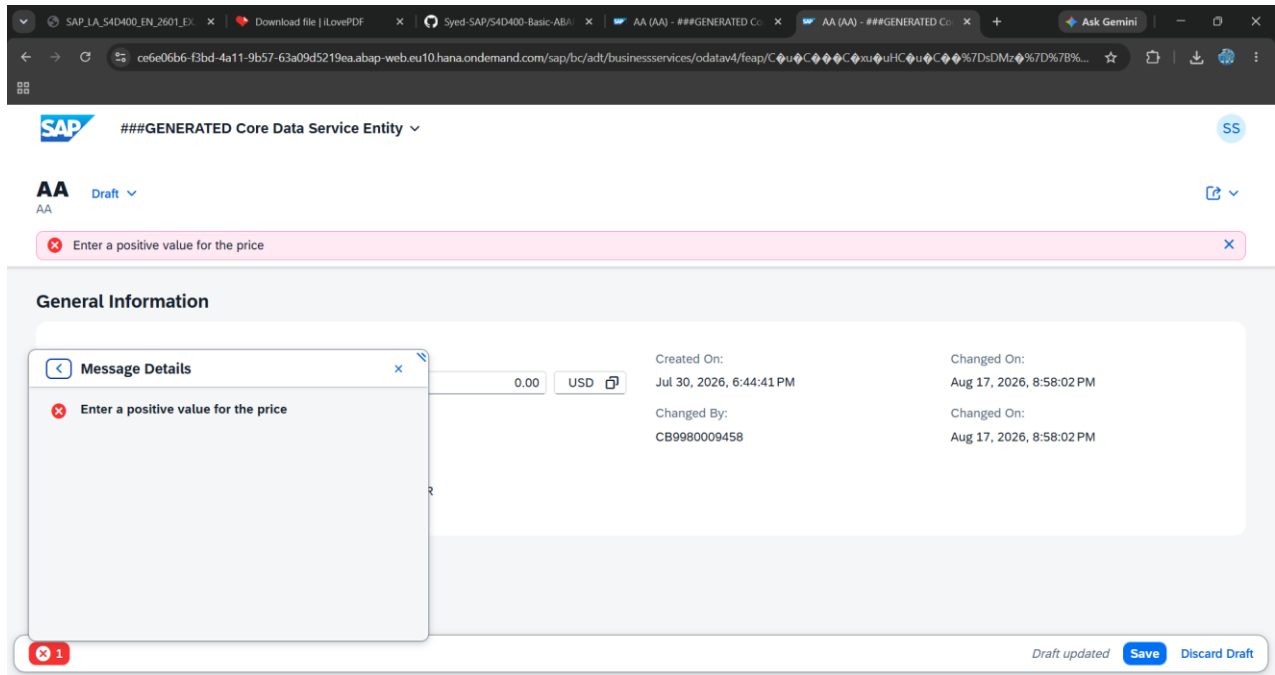
Screenshot 2 – Debugger inside validatePrice after the invalid-price branch



Screenshot 3 – Debugger after the validation message was prepared

12. Invalid Price Test – 0.00

The flight price was changed to 0.00 and Save was selected. The validation correctly rejected the change.

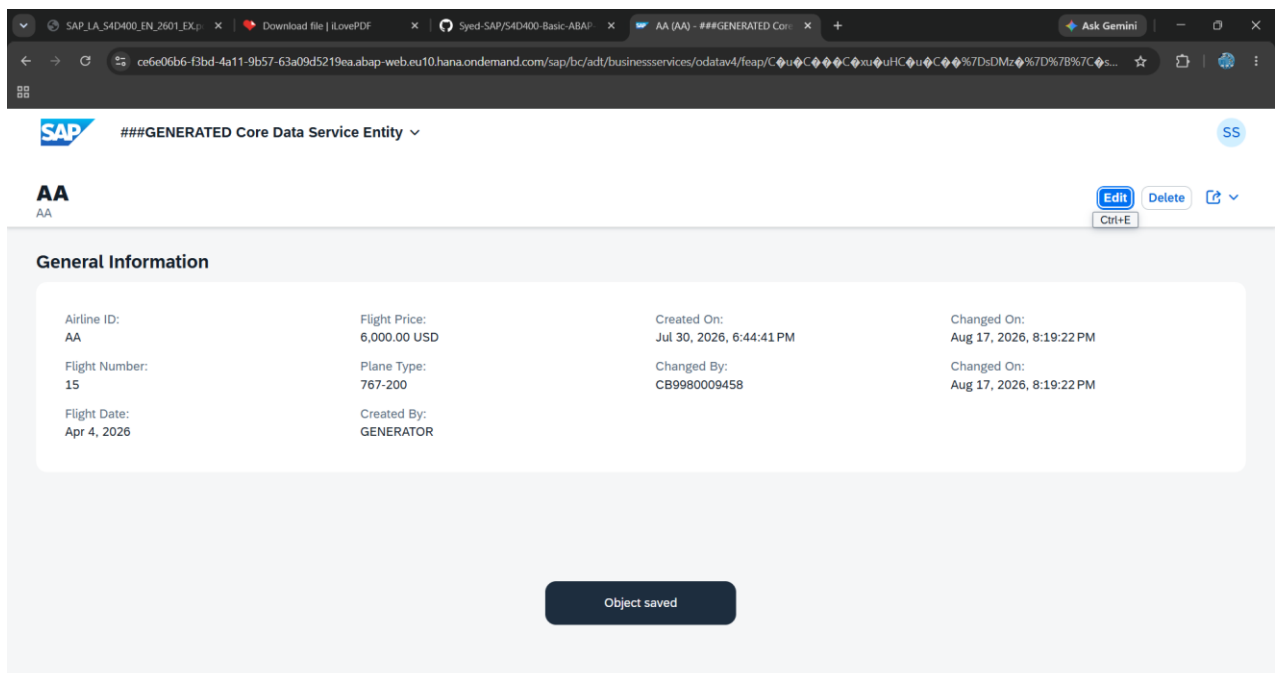


Screenshot 4 – Validation error: Enter a positive value for the price

The Fiori Elements application displays the message 'Enter a positive value for the price'. The draft remains unsaved, confirming that the validation prevented the invalid value from being persisted.

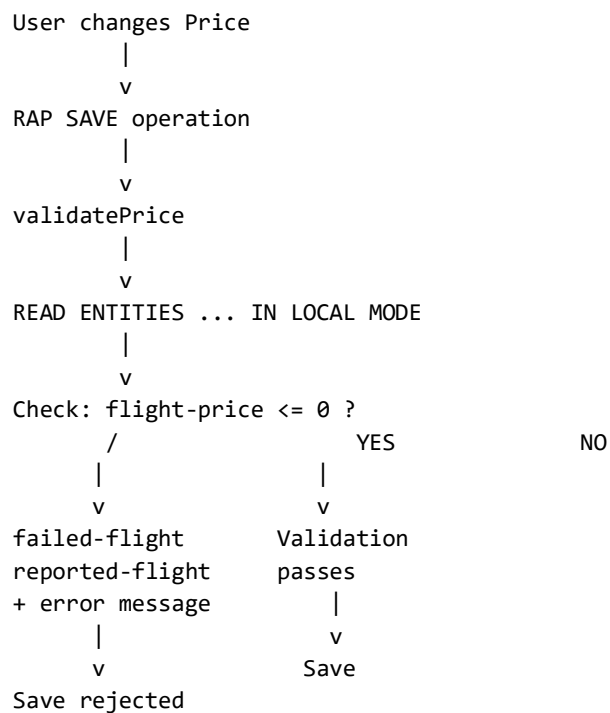
13. Valid Price Test – 6000.00

The price was then changed to 6000.00 and saved. The application displayed 'Object saved', confirming that a positive price passes the validation.



Screenshot 5 – Successful save with a positive flight price

14. Validation Flow



15. Important RAP Concepts Practiced

Concept	Meaning
Validation	Business rule executed during save to prevent invalid data.
ON SAVE	The validation is evaluated during the save processing of the RAP transaction.
IN LOCAL MODE	Reads data from the current RAP transactional buffer rather than relying only on persisted database data.
READ ENTITIES	EML statement used to read RAP business object instances.
failed-flight	Identifies instances for which validation failed.
reported-flight	Returns messages and reported information for affected instances.
%tky	Transactional key used especially with draft-enabled RAP objects.
new_message	Creates a RAP message object for the reported validation error.
ms-error	Marks the message as an error, causing the invalid change to be rejected.

16. Final Verification

Check	Status
Behavior definition contains validatePrice	PASS
Validation implemented in LHC_ZR_09FLIGHT	PASS
READ ENTITIES uses IN LOCAL MODE	PASS
Price <= 0 is rejected	PASS

Error message /LRN/S4D400 101 is reported	PASS
0.00 test displays validation error	PASS
Positive price test saves successfully	PASS

Exercise 20 – COMPLETED

Result: The flight-price validation is implemented, activated, debugged, and tested. Zero and negative prices are rejected, while positive prices are accepted.